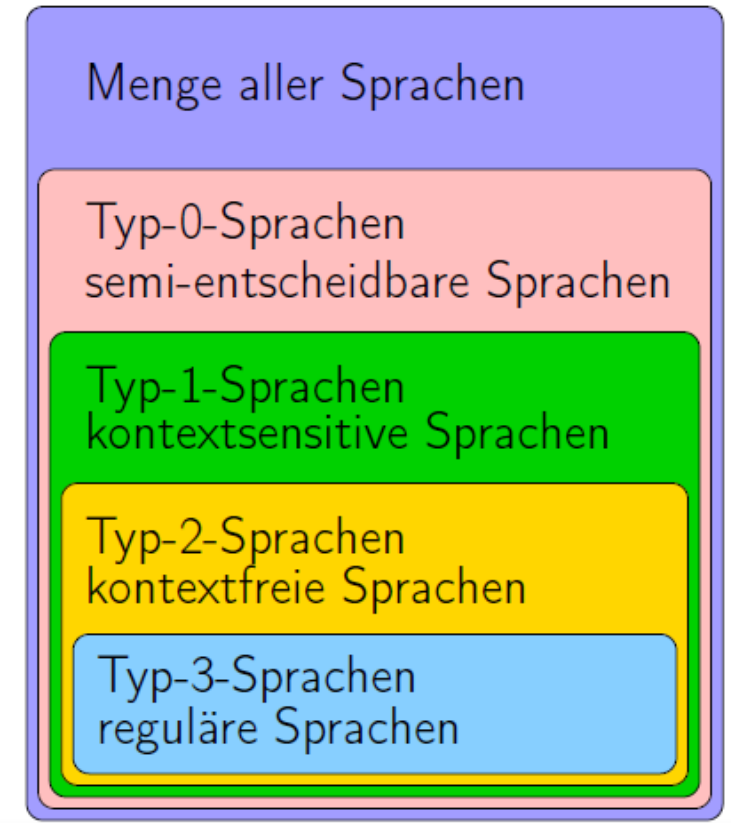




Endliche Automaten

Motivation

- Wir werden gesehen, dass wir mit Computern Typ-0 Sprachen erkennen können.
- Wir werden einfachere Maschinenmodelle kennenlernen, die z.B. den Typ-2 und Typ-3 Sprachen entsprechen → Automaten



Deterministische, endliche Automaten

- Ein **deterministischer, endlicher Automat (deterministic, finite automata; DFA)** ist ein 5-Tupel $(Q, \Sigma, q_0, \delta, F)$, wobei
 - Q die endliche **Zustandsmenge** ist,
 - Σ das endliche **Eingabealphabet** ist,
 - q_0 der **Anfangszustand** ist,
 - $\delta: Q \times \Sigma \rightarrow Q$ die **Zustandsüberföhrungsfunktion** ist und
 - $F \subseteq Q$ die Menge der **akzeptierenden Zustände** ist.

Deterministische, endliche Automaten

Beispiel:

- Es sei $L \subseteq \{0,1\}^*$ die Sprache aller Wörter mit gerade vielen Einsen und gerade vielen Nullen.
- Wie kann eine Maschine aussehen, die diese Sprache akzeptiert?
- Was müssen wir uns nach dem Lesen des Anfangs eines Wortes merken? Ob die Zahl z_0 der gelesenen Nullen gerade und ob die Zahl z_1 der gelesenen Einsen gerade ist. Es werden daher vier Zustände genügen:
 - q_0 : z_0 gerade und z_1 gerade. Dies ist der Anfangszustand, da das leere Wort gerade viele Nullen und gerade viele Einsen enthält. Außerdem ist dieser Zustand akzeptierend.
 - q_1 : z_0 gerade und z_1 ungerade.
 - q_2 : z_0 ungerade und z_1 gerade.
 - q_3 : z_0 ungerade und z_1 ungerade.

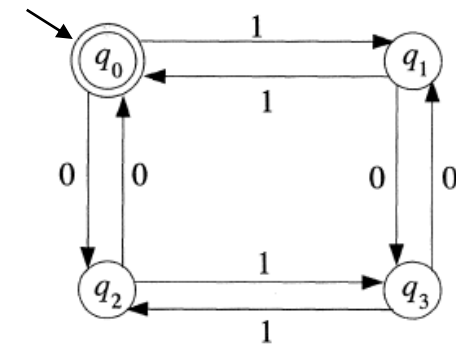
Deterministische, endliche Automaten

Beispiel (cont.):

- Es ist also $Q = \{q_0, q_1, q_2, q_3\}$, $\Sigma = \{0,1\}$, $F = \{q_0\}$ und δ gegeben durch:

δ	q_0	q_1	q_2	q_3
1	q_1	q_0	q_3	q_2
0	q_2	q_3	q_0	q_1

- Grafische Darstellung:
akzeptierende Zustände doppelt eingekreist
Anfangsknoten oft durch einen Pfeil markiert



Deterministische, endliche Automaten

- Grafische Darstellungen sind Standard
- Oft geht nicht von jedem Knoten eine Kante aus.
- Diese fehlenden Kanten interpretieren wir als Kanten zu einem nicht-akzeptierenden Zustand, von dem alle Kanten auf sich selber zeigen.

Deterministische, endliche Automaten

- Setze δ fort nach $\delta: Q \times \Sigma^* \rightarrow Q$.
- Für $q \in Q$ und $w \in \Sigma^*$ soll $\delta(q, w)$ den Zustand angeben, in den der DFA gelangt, wenn er in Zustand q beginnend das Wort w von links nach rechts abgearbeitet hat.
 - Für das leere Wort ϵ sei $\delta(q, \epsilon) := q$.
 - Für $w \in \Sigma$ ist $\delta(q, w)$ bereits gegeben.
 - Für $w = w_1 \dots w_n \in \Sigma^*$ definieren wir $\delta(q, w) = \delta(\delta(q, w_1 \dots w_{n-1}), w_n)$.
- Damit können wir die Akzeptanzregel für einen DFA einfach formal beschreiben. Die Eingabe w wird genau dann akzeptiert, wenn $\delta(q_0, w) \in F$ ist.

Deterministische, endliche Automaten

- DFAs sind offensichtlich weniger mächtig als Turingmaschinen, da Sie nur in linearer Zeit arbeiten können.
- Kann nicht mehr berechnen als eine DTM, da es eine eingeschränkte DTM ist.
- Sie sind aber in einfachen Anwendungen sehr häufig.
- Die von DFAs erkannten Sprachen sind die **regulären Sprachen**. (Typ-3 Sprachen)

Typische Fragestellungen für endliche Automaten bzw. Reguläre Sprachen

- Die Minimierung der Rechenzeit ist für endliche Automaten kein Thema, da die Rechenzeit für ein Eingabewort der Länge n nach Definition n beträgt.
- Versuchen, die Größe des Automaten (=Zustandsmenge) zu minimieren.
 - Dies wird uns mit Hilfe eines effizienten Algorithmus gelingen.
- Welchen Gewinn bringen nichtdeterministische endliche Automaten?
 - Können wir mehr Probleme lösen?
 - Können wir Probleme mit erheblich kleineren Automaten lösen?
- Wie können wir für Probleme zeigen, dass sie nicht durch endliche Automaten lösbar sind?
- Wie kann ich die von Automaten erkannten Sprachen sonst beschreiben?

Minimierung von DFAs

- Eines unserer zentralen Ziele besteht darin, Algorithmen zu optimieren.
- Dies kann für spezielle Probleme schwierig sein. Für die meisten wichtigen Probleme, für die Algorithmen bekannt sind, sind noch keine optimalen Algorithmen bekannt. (Wichtige Ausnahme: Sortierung durch Vergleichen)
- Nach dem Satz von Rice gibt es keinen Algorithmus, der als Eingabe Algorithmen akzeptiert und als Ausgabe einen optimalen Algorithmus für dieselbe Aufgabe liefert.
- Für die äußerst eingeschränkte Klasse der endlichen Automaten, bei denen wir die Güte eines „Algorithmus“ durch die Zahl der Zustände des DFA messen, gibt es allerdings sogar einen effizienten Algorithmus, der für einen DFA A einen äquivalenten DFA A' minimaler Größe berechnet.
- Dabei heißt A' **äquivalent** zu A , wenn A und A' dieselbe Sprache entscheiden. Offensichtlich induziert dies eine Äquivalenzrelation.

Minimierung von DFAs

Zustände eines DFA, die vom Anfangszustand aus nicht erreichbar sind, heißen **überflüssig**.

Satz: Die Menge der überflüssigen Zustände eines DFA kann in Zeit $O(|Q||\Sigma|)$ berechnet werden.

Beweis: Wir stellen endliche Automaten durch gerichtete Graphen mit Kantenbewertungen aus Σ dar. Für jeden Weg im Graphen gibt es eine Eingabe, für die genau dieser Weg durchlaufen wird. Die Menge der nicht überflüssigen Zustände ist also genau die Menge der Knoten im Graphen, die wir mit einer Depth First Suche vom Anfangszustand aus finden. Da jeder der $|Q|$ Knoten des Graphen genau $|\Sigma|$ ausgehende Kanten hat, kann die Depth First Suche in Zeit $O(|Q||\Sigma|)$ durchgeführt werden.

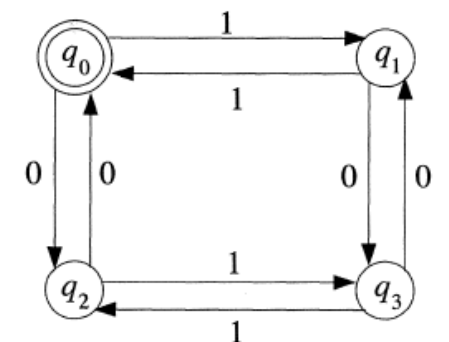
QED

Minimierung von DFAs

- Wir erhalten natürlich einen äquivalenten endlichen Automaten, wenn wir alle überflüssigen Zustände ersatzlos streichen.
- Oft ist es anschaulich hilfreich, zunächst überflüssige Zustände einzufügen.

Minimierung von DFAs

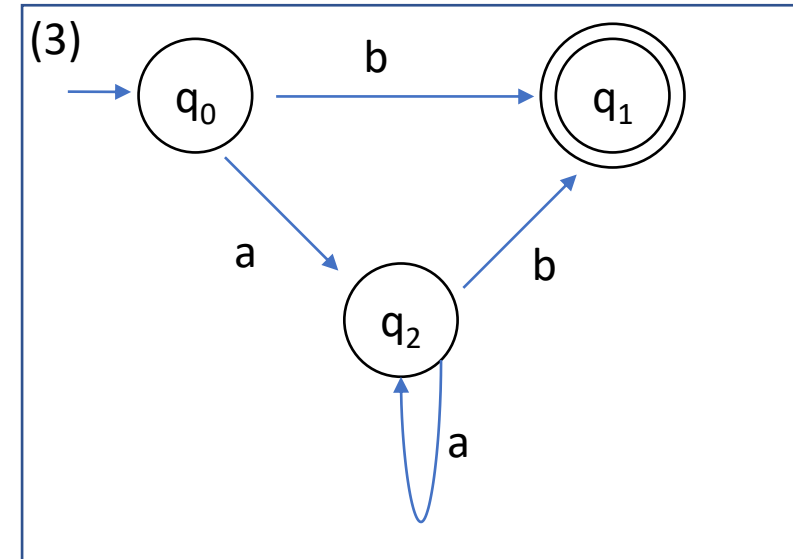
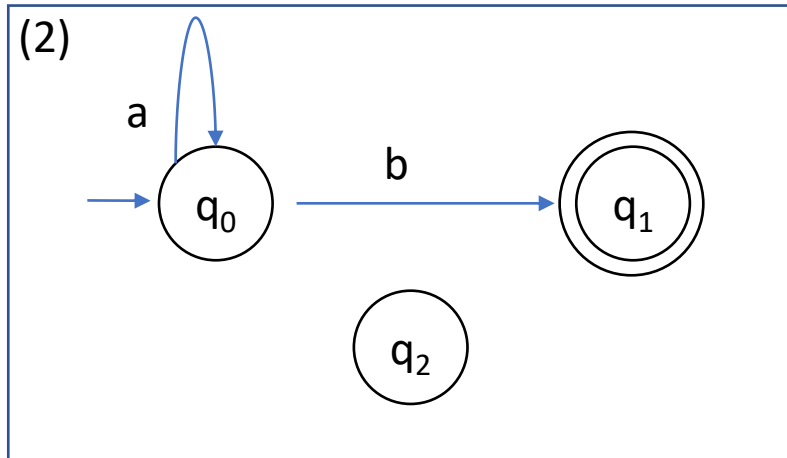
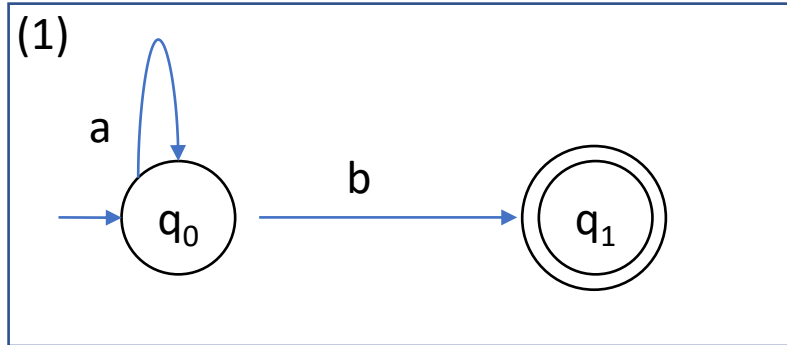
- DFAs ohne überflüssige Zustände sind nicht zwingend minimal.
- Es ist einfach, den gezeigten DFA so zu verändern, dass er mehr als 4 Zustände enthält und keiner der Zustände überflüssig ist.
 - Betrachte zum Beispiel einen DFA mit 16 Zuständen (i, j) mit $0 \leq i, j \leq 3$ und der Interpretation, dass der Zustand (i, j) erreicht wird, wenn für die Zahl z_0 der gelesenen Nullen und die Zahl z_1 der gelesenen Einsen gilt, dass $i = z_0 \bmod 4$ und $j = z_1 \bmod 4$.
- Wie können wir einen derartigen DFA minimieren?
- Fasse „äquivalente“ Zustände zusammenfassen.



Beispiel

Startknoten q_0

$w = a^n b, n \geq 0$



(1) : minimaler DFA

(2) : DFA mit überflüssigem Zustand q_2

(3) : DFA ohne überflüssige Zustände, aber nicht minimal

Minimierung von DFAs

- Zwei Zustände sind für das Akzeptanzverhalten des Automaten genau dann äquivalent, wenn es für das Akzeptanzverhalten unerheblich ist, ob wir in dem einen oder dem anderen Zustand starten.
- Zwei Zustände p und q eines DFA heißen **äquivalent**, Notation $p \equiv q$, wenn für alle Wörter $w \in \Sigma^*$ gilt: $\delta(p, w) \in F \Leftrightarrow \delta(q, w) \in F$.
- Mit $[p]$ bezeichnen wir die **Äquivalenzklasse** der zu p äquivalenten Zustände.

Minimierung von DFAs

Der **Äquivalenzklassenautomat** A' zu einem DFA A hat folgendes Aussehen: $Q' = \{[q] | q \in Q\}$, $\Sigma' = \Sigma$, $q'_0 = [q_0]$, $F' = \{[q] | q \in F\}$, $\delta([q], w) = [\delta(q, w)]$.

Satz: Der Äquivalenzklassenautomat A' ist wohldefiniert und akzeptiert die gleiche Sprache wie A .

Was bedeutet Wohldefiniert?

Beweis: Wegener, Satz 4.2.5.

Minimierung von DFAs

- Nach Kenntnis der Äquivalenzklassen bezüglich „ $\equiv$ “ kann der Äquivalenzklassenautomat in Zeit $O(|Q||\Sigma|)$ konstruiert werden.
- Wie berechnet man Äquivalenzklassen?
- In der Definition steht ein Allquantor („ $\forall$ “) über eine unendliche Menge. Existenzaussagen sind leichter zu verifizieren als Allaussagen. Die Nichtäquivalenz zweier Zustände lässt sich durch eine Existenzaussage beschreiben:

$$p \not\equiv q \Leftrightarrow \exists w \in \Sigma^*: \delta(p, w) \in F \text{ und } \delta(q, w) \notin F$$

$$\text{oder } \delta(p, w) \notin F \text{ und } \delta(q, w) \in F$$
- Ein derartiges Wort w nennen wir einen **Zeugen** für die Nichtäquivalenz von p und q .

Minimierung von DFAs

- Betrachte das Verhalten bei Start in $p \in Q$ und bei Start in $q \in Q$ für alle Zustandspaare (q, p) gleichzeitig.
- Definieren das **Automatenprodukt** von A mit sich selbst. Dieser DFA A' arbeitet auf der Zustandsmenge $Q \times Q$ und hat Zustandsüberföhrungsfunktion

$$\delta'((p, q), a) := (\delta(p, a), \delta(q, a))$$
- Einfache Übung: definiere Anfangszustand und akzeptierende Zustände.
- Die Menge I inäquivalenter Zustandspaare enthält auf jeden Fall $F \times (Q - F)$ und $(Q - F) \times F$, da für sie die Inäquivalenz durch das leere Wort bezeugt wird.
- Von allen inäquivalenten Zustandspaaaren (p, q) führt jeder Zeuge in A' in einen dieser Zustände.
- Also werden genau die inäquivalenten Zustandspaare bei einer Rückwärtssuche von $F \times (Q - F) \cup (Q - F) \times F$ gefunden.

Minimierung von DFAs

Algorithmus (Minimierung von DFAs)

Eingabe: DFA A .

Ausgabe: Menge der Paare inäquivalenter Zustände.

- 1.) Bilde das Automatenprodukt A' von A mit sich selbst.
- 2.) Kehre in dem Graphen zu A' die Richtung aller Kanten um. Der entstehende Graph sei G .
- 3.) Berechne in G alle Knoten, also Zustandspaare, die von $F \times (Q - F) \cup (Q - F) \times F$ erreichbar sind (zB mit Depth First Suche).

Satz: Der Algorithmus berechnet die Menge inäquivalenter Zustandspaare in $O(|Q|^2|\Sigma|)$ Schritten.

Beweis: Wegener, 4.2.7. (Es geht sogar in $O(|Q||\Sigma|\log |Q|)$)

QED

Minimierung von DFAs

Satz: Wenn aus einem DFA A zunächst die überflüssigen Zustände entfernt werden und dann daraus der Äquivalenzklassenautomat A' konstruiert wird, ist A' ein minimaler zu A äquivalenter endlicher Automat.

Beweis: Wegener, 4.2.13 (und Theorie zuvor)

QED

Tools zur Arbeit mit DFAs

- Diverse Tools im Internet zur Erzeugung von DFAs

- FLACI

FLACI

Formale Sprachen, abstrakte Automaten, Compiler und Interpreter

DIE MODULE

Formale Sprachen

Ein Alphabet A ist eine endliche, nichtleere Menge von Zeichen.

Wählen Sie eines der Beispiel-Alphabete zum Experimentieren aus:

- ☐ $A_1 = \{0, 1\}$
- ☐ $A_2 = \{a, b, c, \dots, z\}$
- ☐ $A_3 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- ☒ $A_4 = \{ \text{☹, ☺, ☹☺, ☹☹☹, ☹☹☹☹} \}$
- ☐ $A_5 = \{ \text{begin, end, for, while, do, repeat, until} \}$

Interaktives Minututorial zu den Grundbegriffen formaler Sprachen

Reguläre Ausdrücke

Der einfachste reguläre Ausdruck a steht für ein einzelnes Wort "a", kurz: a . Es besteht aus genau einem Alphabetzeichen a . Der Ausdruck beschreibt die Sprache $L(a)$. Reguläre Ausdrücke lassen sich verkettend a gefolgt von b , gefolgt von c oder kurz abc schreiben.

Wie ändert sich die beschriebene Sprache, wenn man den regulären Ausdruck a zu ab oder abc verändert?

Regulärer Ausdruck: a

Originalsprache: a

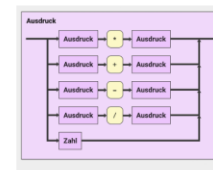
ab

abc

Spezies Diagramm: $\rightarrow a \rightarrow$

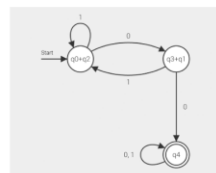
Interaktives Minututorial zu regulären Ausdrücken

Formale Grammatiken



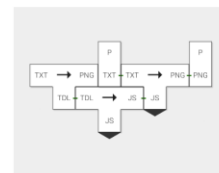
Kontextfreie Grammatiken entwickeln, transformieren und konvertieren

Abstrakte Automaten



Abstrakte Automaten konstruieren, simulieren, transformieren und konvertieren

Compiler und Interpreter



Modellieren von Übersetzungsprozessen und Entwicklung von Compilern und Interpretern

FLACI

Abstrakte Automaten

Meine Automaten

Beispiel DEA

Definition

Alphabet

Ausrichten

Überprüfen

Simulation

Transformieren

Konvertieren

Ein *deterministischer endlicher Automat*, kurz: DEA (oder EA), ist durch ein 5-Tupel $M = (Q, \Sigma, \delta, q_0, E)$ definiert.

Die verwendeten Symbole haben folgende Bedeutungen:

Q ... endliche Menge der Zustände

Σ ... Eingabealphabet

δ ... totale Überföhrungsfunktion $Q \times \Sigma \rightarrow Q$

q_0 ... Anfangszustand, $q_0 \in Q$

E ... Menge von Endzuständen, $E \subseteq Q$

Der Automat akzeptiert bei Erreichen eines Endzustands am Wortende.

6 als partielle Funktion erlauben

Start

z1

z2

z3

z4

z5

c

a

b

a, b, c

b

a, b, c

a

a

a, b, c

a, b, c

Eingabewort

c a b c

Simulation starten

Langsam

Schnell

Letzte Eingabewörter:

Konfigurationenfolge(n) für: